Bachelor of Data Science

Module
PGR107 Python Programming

Due date for submission
Spring 2026(see Wiseflow)

Module leader and e-mail
Huamin Ren | Huamin.Ren@kristiania.no

Teacher and e-mail
Huamin Ren | Huamin.Ren@kristiania.no
Miroslav Bachinski Miroslav.Bachinski@kristiania.no

Learning outcomes

After successfully completing the course the student:

Knowledge

- has an understanding of basic Python programming skills for basic data structure, data manipulation, data visualization and data analysis on various data formats
- are familiar with programming paradigms such as object oriented programming, functional programming
- can properly choose the suitable data structures and manipulation methods to map the real world problems into the Python programming world; can perform data preparation, data transformations and data visualization to provide a solution

Skills

- can demonstrate an understanding of the underlying data
- can write simple programs in Python programming language that make use of basic data structure, expression and control flows
- can implement more complicated programming using Python external libraries through problem mapping
- can create data visualizations using open-source libraries in Python

General competence

- can identify the key challenges and design goals for performing data analysis using Python programming
- can critically assess the benefits of using Python data analytics programming
- can demonstrate basic Python coding skills for data manipulation, data analysis, and data visualization on various data formats

Assignment specification

Exam: Written home examination individually or in group (2-4 students)
Code implementation with comments (note: use %% md for block-wise comments), should be saved and submitted as .ipynb (share directly on wise flow).
Referencing: Any acceptable academic style.

Please address the following questions in your submission.

1. (15 points) The (fictional) Supernacci numbers s(n) can be computed according to the following rules:

- s(1) = 1,
- s(n) is the sum of the last (n - 1) % 3 Supernacci numbers, or s(n-1) if the sum is empty.

Write a function supernacci(n), which computes and outputs the n-th Supernacci number. Use slicing and the sum(list) function instead of recursion.

2. (17 points) You meet an online gambler who wants to play a game with you. He has a 4-sided dice (You can Google 4-sided dice for an example), and you need to guess the number rolled. He is not a very good gambler, so he allows you to try to guess up to 4 times (till the correct guess). He is also a novice in Python, so his application has multiple bugs. He provides the source code for the application below. You need to find all the bugs and fix the code.

```
import random as rnd

dice = 0
guess = 0
def getDiceGuess():
    guesss = 0
    while guesss not in ('1 ','2 ','3 ','4 '):
        print('Guess the number on a 4- sided dice! Enter a number
between 1 and 4:')
    guess = input()
    return guesss

def rollDice():
    dice = random.randint(1, 5) # count of dots on a 4-sided dice

if dice == guess:
    print('You are good!')
else:
    print('Sorry! Try to guess again!')
    guesss = input()
    if dice = guess:
        print('Congrats! You got it!')
    else:
        print('Sorry! Your third chance!')
        guess = input()
        if dice = guess:
            print('Congrats! You got it!')
        else:
            print('Sorry! You still can try!')
        guesss = input()
        if dice = guess
            print('Congrats! You got it!')
        else:
            print('Nope. You are really bad at this game.')
```

3. (30 points) You are given two Python dictionaries as input:

The first dictionary contains city names as keys. The value for each city is a list containing:

- the latitude and longitude of the city center (in degrees),
- the city size, defined as the distance from the center to the farthest point of the city (in km).

The second dictionary contains geographical coordinates (latitude, longitude) as keys. The value for each key is a list containing:

- the name of a place of interest (POI),
- its customer rating.

Example inputs:

```
cities = {'Bergen': [60.364098, 5.404422, 15],
          'Oslo': [59.926959, 10.779454, 10],
          'Trondheim': [63.433654, 10.390277, 5]
}
poi = {(63.426806, 10.396712): ['Nidarosdommen', 4.5],
       (59.926699, 10.701075): ['Vigelandsparken', 5],
       (59.907704, 10.753163): ['Operahuset', 4],
       (60.397561, 5.322833): ['Bryggen', 4.5],
       (60.387639, 5.321623): ['Universitetmuseet', 5],
       (60.342901, 5.336796): ['Gamlehaugen', 4]
}
```

Use the following approximations:
1 degree of latitude = 111 km
1 degree of longitude = 68 km

a) (6 points) Find the nearest city and POI

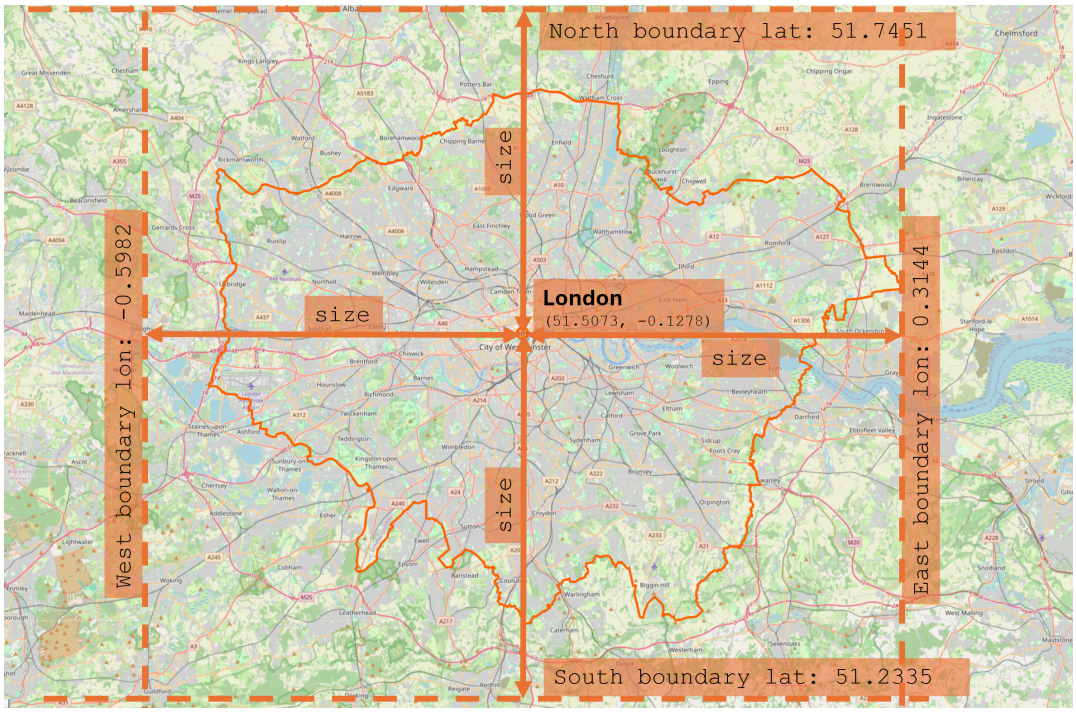
Write:

- one function that returns the closest POI for a given pair of coordinates,
- one function that returns the closest city for a given pair of coordinates.

Then ask the user to enter a latitude and longitude, and print the closest city and closest POI.

b) (6 points) Compute city boundaries

Extend the city data by computing the city boundaries:
north,
south,
east,
west.



Assume that each city has a square shape and that the city size can be added from the center in each direction, as you can see in the image. Ignore the curvature of the Earth when working with latitude and longitude.

c) (6 points) Assign POIs to cities

For each city, determine which POIs are located inside its boundaries.

A POI belongs to a city if:

- its latitude lies between the south and north boundaries,
- its longitude lies between the west and east boundaries.

d) (6 points) Compute average POI rating per city

For each city, compute the average customer rating of all POIs located in that city.

e) (6 points) Order POIs by distance from the city center

For each city, create a list of POI names ordered by their distance from the city center, from nearest to farthest.

4. (8 points) Implement a function trace(A) that takes a rank 2 numpy array A (i.e., a matrix) and checks whether the number of columns coincide with the number of rows. If (and only if) these numbers coincide, it calculates the sum of elements on the main diagonal of A and outputs the result. Do not limit the code to the example below.
Example:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Expected Output: 1

5. (30 points) A use-case study.

Norwegian high school grade statistics provide detailed information about standpunkt, written exam, and oral exam results across schools, counties, and the national level. See the link below:

> https://www.udir.no/tall-og-forskning/statistikk/statistikk-videregaende-skole/karakterer-vgs/

The data makes it possible to study grading patterns across time, geography, subjects, and assessment forms.

Browsing the link (above) and also research on other relevant resources, understand the left columns indicating school years (skoleår), character type (karaktertype) – indicating the grades containing oral exam, written exam and so on. Search on other relevant webpages to understand the grading system in high school in Norway and what each exam refers to, for example understand 'standpunkt', 'muntlig eksammen' and many other relevant terms on the webpage. Then provide the implementation to answer the following questions.

5.1 (10 points) Navigate the webpage, then choose high schools from Oslo areas (hint: from 'Enhet'), school year 2024-25, subject, and assessment type, then download the file as .CSV file. You can choose all subjects or some subjects, but later in the analysis you need to address the reason why you made such selection. The same also applies to other features for example assessment types. Implement using python to load the .CSV file, convert to proper data types.

5.2 (10 points) Explore the data you processed in step 2.1 with proper visualization. Explain your finding after visualization.

5.3 (10 points) Define a specific problem and implement a Python-based solution. For example, you can define a similar or exact problem as the following, but be noted that you are not limited to such definition.

> *For example:*
>
> *There has been heat discussion in public regarding the systematic gaps between standpunkt and muntlig eksammen. Udir shows some analytics indicating that written exam grades are often lower than standpunkt grades, while oral exam grades are often higher.*
>
> *Aftenposten has an overview of the standing grades vs. exam grades for all Norwegian high schools. https://www.aftenposten.no/norge/i/3j7pM/se-om-skolen-din-er-streng-eller-snill-med-karakterene?utm_source=chatgpt.com*
>
> *Define the problem as: Based on the historical performance of schools from Oslo areas from year 2020-2025, if the grades have similar pattern as in the article or not? How about other regions/counties/cities? Then show your findings with proper implementation, and conclude your findings.*

Assignment criteria*

Grade	Learning Outcome 1: Knowledge	Learning Outcome 2: Skills	Learning Outcome 3: Competence
A Excellent	Excellent and comprehensive understanding of concepts	Demonstrates excellent analytical, technical and writing skills	Outstanding degree of judgment and independent critical thinking
B Very good	Very good understanding of concepts	Demonstrates very good analytical, technical and writing skills	Sound degree of judgment and independent critical thinking
C Good	Good understanding of theory in most important areas	Demonstrates good analytical, technical and writing skills	Reasonable degree of judgment and independent critical thinking
D Satisfactory	Satisfactory understanding of theory, but with significant shortcomings	Demonstrates limited analytical, technical and writing skills	Limited degree of judgment and independent critical thinking
E Sufficient	Meets the minimum understanding of concepts	Demonstrates sufficient analytical, technical and writing skills	Very limited degree of judgment and independent critical thinking
F Fail	Fail to meet the minimum academic criteria.	No demonstration of analytical, technical and writing skills	Absence of judgment and independent critical thinking

*Adapted from The Norwegian Association of Higher Education Institutions

The assignment is worth 100 % of the grade of the course.